MLOps Assignment 1 - Instructions to set up and run

Author: Rouzif Rasheed Moopan, 2025AE05986

Step 1: Prerequisites & Environment Setup

Ensure the target machine has the following dependencies installed prior to execution:

- **Git** (v2.30+)
- **Python** (v3.10+)
- **Docker Desktop** (with Kubernetes enabled) or **Minikube**
- **kubectl CLI**
    - Download the latest stable release binary
      `curl -LO "https://dl.k8s.io/release/$(curl -L -s https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"`
    - Make the binary executable
      `chmod +x kubectl`
    - Move the binary to /usr/local/bin
      `sudo mv kubectl /usr/local/bin/`
    - Check installation
      `kubectl version --client`

Step 1.1: Clone the Repository

Retrieve the project source code and navigate into the working directory:

```
git clone https://github.com/rouzif/WILP_MLOps_Assignment_1.git
cd WILP_MLOps_Assignment_1
```

Step 1.2: Create and Activate Virtual Environment

Initialize an isolated Python virtual environment:

- **macOS / Linux:**
  ```
  python3 -m venv .venv
  source .venv/bin/activate
  ```

- **Windows (PowerShell):**
  ```
  python -m venv .venv
  .venv\Scripts\Activate.ps1
  ```

Step 1.3: Install Dependencies

Upgrade the core package manager and install all required runtime and development packages:

```
pip install --upgrade pip
pip install -r requirements.txt
```

Step 2: Local Testing & ML Pipeline Execution

Step 2.1: Run Unit Tests

Execute the automated test suite to ensure system integrity:

```
python -m pytest tests/ -v
```

Step 2.2: Execute Training Pipeline

Run the end-to-end model training workflow:

```
python main.py --pipeline train
```

Step 2.3: (Optional) Launch MLflow Tracking Server

To inspect experiment logs, run parameters, and performance metrics visually:

```
mlflow server --backend-store-uri sqlite:///data/mlflow.db
--default-artifact-root ./mlruns --host 127.0.0.1 --port 5001
```

Access URL: Navigate to `http://localhost:5001` in your web browser.

---

3. Docker Containerization

Step 3.0: Navigating to working directory in new terminal

Open a new terminal and navigate to working directory

```
cd WILP_MLOps_Assignment_1
```

Step 3.1: Build Container Image

Package the application into a standard container image:

```
docker build -t heart-disease-api:v1 .
```

Step 3.2: Run Container Locally

Instantiate the container service in detached mode:

```
docker run -d --rm --name heart-disease-service -p 8000:8000
heart-disease-api:v1
```

Step 3.3: Verify Endpoints

Validate container operational readiness using the following interfaces:

- **Health Probe:**
  `curl -X GET http://localhost:8000/health`
- **Inference Endpoint:**
  `curl -X POST http://localhost:8000/predict -H "Content-Type: application/json"  -d '{"age": 52, "sex": 1, "cp": 0, "trestbps": 125, "chol": 212, "fbs": 0, "restecg": 1, "thalach": 168, "exang": 0, "oldpeak": 1.0, "slope": 2, "ca": 2, "thal": 3}'`

- **Swagger API Documentation:**
  Access interactive API docs at `http://localhost:8000/docs`

Step 3.4: Stop Local Container

Terminate the container instance prior to Kubernetes deployment:

```
docker stop heart-disease-service
```

---

4. Kubernetes Deployment

Step 4.1: Enable Cluster

Ensure your local Kubernetes engine (Docker Desktop Kubernetes or Minikube) is actively running:

```
minikube start
kubectl cluster-info
```

Step 4.2: Build docker image directly inside Minikube's docker daemon

```
eval $(minikube docker-env)
docker build -t heart-disease-api:v1 .
```

Step 4.3: Apply Kubernetes Manifests

Deploy the namespace, application pods, and routing service:

```
kubectl apply -f k8s/namespace.yaml
kubectl apply -f k8s/deployment.yaml
kubectl apply -f k8s/service.yaml
```

Step 4.4: Verify Cluster Resources

Confirm that all resources have been instantiated and pods are in a `Running` state:

```
kubectl get pods,svc,deploy -n heart-disease-mlops
```

Step 4.5: Port-Forward API Service

Expose the Kubernetes service to your local workstation:

```
kubectl port-forward svc/heart-disease-api-service 8000:8000 -n
heart-disease-mlops
```

---

5. API Access and Endpoint testing

Health Probe:

```
curl -X GET http://localhost:8000/health
```

Inference Endpoint:

```
curl -X POST http://localhost:8000/predict -H "Content-Type: application/json"
-d '{"age": 52, "sex": 1, "cp": 0, "trestbps": 125, "chol": 212, "fbs": 0,
"restecg": 1, "thalach": 168, "exang": 0, "oldpeak": 1.0, "slope": 2, "ca": 2,
"thal": 3}'
```

Swagger API Documentation:

Access interactive API docs at `http://localhost:8000/docs`.